

ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ПРОФЕССИОНАЛЬНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ИРКУТСКОЙ ОБЛАСТИ
«АНГАРСКИЙ ТЕХНИКУМ СТРОИТЕЛЬНЫХ ТЕХНОЛОГИЙ»

Курсовая работа

Тема: Разработка сервиса для управления складом предприятия

Разработал _____ Прокопенко А.В

Руководитель _____ Денисюк А.В

Ангарск, 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
1. ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ	10
1.1. Анализ систем складского учёта и WMS-решений	10
1.1.1. Основные функции складского учёта	11
1.1.2. Экономическая эффективность автоматизации	13
1.2. Выбор операционной системы в качестве платформы	14
1.2.1. Microsoft Windows как основная платформа разработки и эксплуатации	15
1.2.2. Обоснование итогового выбора	17
1.3. Обоснование выбора языка программирования и фреймворка	18
1.3.1. Обоснование выбора Python 3.x	18
1.3.2. Интеграция с фронтенд-стеком (HTML5, CSS3, Vanilla JS)	20
1.4. Среда разработки и инструменты отладки	20
1.4.1. Интегрированные среды разработки (IDE) для Python	21
1.4.2. Инструменты отладки и профилирования	22
1.4.3. Управление зависимостями и виртуальные окружения	24
1.5. Архитектура и технологический стек решения	25
1.5.1. Клиент-серверная архитектура с тонким клиентом	25
1.5.2. Обоснование стека для целевой среды эксплуатации	27
2. РАЗРАБОТКА СЕРВИСА ДЛЯ УПРАВЛЕНИЯ СКЛАДОМ	28
2.1. Структура проекта и организация файловой системы	28
2.1.1. Детальное описание корневых модулей ядра приложения	29
2.1.2. Обоснование архитектурных решений	31
2.2. Архитектура системы и принципы проектирования	32
2.2.1. Разделение ответственности (Separation of Concerns)	33
2.2.2. Транзакционная целостность и обработка исключений	33
2.3. Реализация функциональных модулей	34

						КР-09.02.07-К-245-26ПЗ					
Изм.	Кол.уч	Лист	№ док.	Подпись	Дата	Разработка сервиса для управления складом предприятия			Лит.	Лист	Листов
Разраб.		Прокопенко А.В.									
Пров.		Денисюк А.В.								3	55
									ГАПОУ ИО АТСТ		
Н.контр											
Утв.											

ВВЕДЕНИЕ

Современная логистика и управление цепочками поставок (supply chain management) находятся в состоянии непрерывной цифровой трансформации. Склад, будучи ключевым узлом материальных потоков любого производственного, торгового или дистрибьюторского предприятия, требует точного, оперативного и надёжного учёта товарно-материальных ценностей. Ошибки в учёте остатков, несвоевременное оформление приходных или расходных документов, отсутствие прозрачной аналитики движения товаров напрямую ведут к финансовым потерям, срыву производственных графиков, затовариванию складских площадей или, напротив, к дефициту критически важных позиций. В условиях высокой конкуренции и ужесточения требований к операционной эффективности предприятий вопрос автоматизации складского учёта перестаёт быть опциональным и становится стратегической необходимостью.

На рынке корпоративного программного обеспечения доминируют масштабные ERP- и WMS-системы (например, 1С:Предприятие, SAP EWM, Microsoft Dynamics 365 Supply Chain Management). Данные решения обладают широким функционалом, глубокими интеграционными возможностями и соответствуют строгим отраслевым стандартам. Однако их внедрение сопряжено со значительными финансовыми затратами: стоимость лицензий, оплата услуг интеграторов, необходимость обучения персонала, регулярные платежи за техническую поддержку и обновления.

Для малых и средних предприятий, микропредприятий, а также для локальных складских подразделений в составе крупных компаний такие решения зачастую оказываются избыточными. Кроме того, тяжёлые системы требуют выделенного ИТ-штата, мощной серверной инфраструктуры и длительного цикла адаптации под специфические бизнес-процессы.

						КР-09.02.07-К-245-26ПЗ	Лист
							5
Изм.	Кол.цч	Лист	№ докум.	Подпись	Дата		

В связи с этим возникает объективная потребность в создании лёгких, модульных и экономически доступных складских сервисов, разработанных на базе открытых технологий. Веб-ориентированная архитектура «тонкий клиент» позволяет развёртывать систему на обычном рабочем компьютере или недорогом сервере, обеспечивая доступ через стандартный браузер с любого устройства в локальной сети или через защищённый интернет-канал. Использование языков и фреймворков с открытым исходным кодом исключает лицензионные отчисления, снижает зависимость от конкретных вендоров и даёт разработчикам полный контроль над эволюцией продукта.

Именно в этом контексте была сформулирована тема данной курсовой работы: «Разработка сервиса для управления складом предприятия». Проект ориентирован на создание полнофункционального веб-приложения, способного закрыть базовые и расширенные потребности складского учёта без привлечения коммерческих платформ. Реализация выполнена на стеке современных открытых технологий: серверная логика построена на языке программирования Python 3.x с использованием микрофреймворка Flask; взаимодействие с данными организовано через ORM-библиотеку SQLAlchemy и встроенную реляционную СУБД SQLite; интерфейс формируется шаблонизатором Jinja2 с применением стандартов HTML5, CSS3 и нативного JavaScript; аутентификация и управление сессиями обеспечиваются расширением Flask-Login.

Актуальность исследования обусловлена следующими факторами:

Рост требований к прозрачности учёта. Современные стандарты бухгалтерского и управленческого отчётности требуют сквозной трассировки движений товаров, фиксации авторства операций и хранения истории изменений. Представленный сервис реализует журнальный принцип учёта (stock ledger), исключая рассинхронизацию остатков и обеспечивающий полный аудит-трейл.

						КР-09.02.07-К-245-26ПЗ	Лист
							6
Изм.	Кол.цп	Лист	№ докум.	Подпись	Дата		

Доступность веб-технологий. Развитие клиент-серверных архитектур, стандартов безопасности (HTTPS, сессионные cookie, RBAC) и инструментов быстрой разработки (microframeworks, ORMс) сделало создание production-ready систем возможным для команд небольшой численности или индивидуальных разработчиков.

Экономическая эффективность для МСБ. Использование бесплатного стека (Python, Flask, SQLite, Jinja2, Vanilla JS) снижает совокупную стоимость владения (ТСО) практически до нуля, оставляя единственной статьёй расходов время разработки и квалификацию специалистов.

Образовательная и практическая ценность. Проект демонстрирует применение инженерных практик: разделение ответственности (Separation of Concerns), транзакционная целостность, декларативное управление доступом, серверный рендеринг с прогрессивным улучшением UX, динамическая аналитика (ABC-классификация, графики движения) и экспорт данных в машиночитаемые форматы (CSV).

Целью данной работы является проектирование и программная реализация веб-сервиса для автоматизации складского учёта, обеспечивающего полный жизненный цикл складских операций: от регистрации номенклатуры и оформления приходно-расходных документов до динамического контроля остатков, формирования регламентных отчётов и аналитики эффективности товарных групп.

Для достижения поставленной цели были решены следующие задачи: Проведён анализ предметной области складского учёта, изучены эволюция WMS-систем, современные архитектурные подходы и экономические обоснования автоматизации.

Выполнено обоснование выбора технологического стека, включая операционную систему развёртывания, язык программирования, веб-фреймворк, систему управления базами данных и средства фронтенд-разработки.

						КР-09.02.07-К-245-26ПЗ	Лист
							7
Изм.	Кол.изм.	Лист	№ докум.	Подпись	Дата		

Спроектирована реляционная структура данных с учётом принципов нормализации, ссылочной целостности и оптимальной индексации для типовых запросов складской аналитики.

Реализована модульная архитектура приложения, включающая подсистемы аутентификации и авторизации, управления номенклатурой, документооборота с валидацией остатков, журналирования движений, формирования отчётов и администрирования.

Внедрены механизмы информационной безопасности: криптографическое хеширование паролей (PBKDF2-SHA256), защищённое хранение сессий, ролевая модель доступа (RBAC), автоматическое экранирование выводимых данных, параметризованные запросы через ORM.

Разработан адаптивный пользовательский интерфейс с поддержкой серверной генерации страниц, клиентской сортировки и фильтрации таблиц, динамической отрисовки графиков на <canvas> и экспорта данных в CSV.

Проведено многоуровневое тестирование: функциональная проверка сценариев, анализ устойчивости к некорректному вводу, проверка ограничений доступа, оценка производительности при последовательных транзакциях.

Подготовлены рекомендации по переводу системы в промышленную эксплуатацию: настройка production-сервера, обеспечение HTTPS, организация резервного копирования, планирование масштабирования.

Практическая значимость разработанного сервиса заключается в его готовности к развёртыванию на реальных объектах малого и среднего бизнеса, а также в возможности использования в качестве учебного полигона для отработки навыков fullstack-разработки на Python. Система не требует установки клиентского ПО, работает в любом современном браузере, поддерживает многопользовательский режим с чётким разделением ролей (администратор, кладовщик, менеджер) и обеспечивает прозрачность всех складских операций. Архитектура приложения заложена

						КР-09.02.07-К-245-26ПЗ	Лист
							8
Изм.	Кол.цм	Лист	№ докум.	Подпись	Дата		

с учётом будущих расширений: добавления интеграции со сканерами штрихкодов, миграции на кластерные СУБД (PostgreSQL), внедрения кэширования (Redis) и контейнеризации (Docker).

Объект исследования — процессы складского учёта, документооборота и аналитики товарных потоков на предприятии.

Предмет исследования — программно-технические средства и архитектурные решения, обеспечивающие автоматизацию указанных процессов в рамках веб-ориентированного сервиса.

Методы исследования, применённые в работе: анализ технической литературы и документации, сравнительный анализ технологических стеков, объектно-реляционное моделирование, модульное и интеграционное тестирование, статический анализ кода, проектирование пользовательского интерфейса по принципам юзабилити и доступности (a11y).

Структура курсовой работы логически выстроена в соответствии с этапами жизненного цикла разработки программного обеспечения. Первая глава посвящена теоретическому обоснованию выбора технологий, анализу современных подходов к складской автоматизации и рассмотрению экономического эффекта от внедрения подобных систем. Вторая глава содержит детальное описание практической реализации: архитектуры проекта, структуры базы данных, алгоритмов проведения документов, механизмов безопасности, пользовательского интерфейса и подготовки к промышленной эксплуатации. В заключении сформулированы выводы о достижении поставленных целей, ограничениях текущей версии и перспективах развития продукта. Работа сопровождается списком использованных источников и приложениями, содержащими фрагменты исходного кода, схему базы данных и скриншоты интерфейса.

1. ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

1.1. Анализ систем складского учёта и WMS-решений

Система управления складом (Warehouse Management System, WMS) представляет собой специализированное программное обеспечение или комплекс программно-аппаратных средств, предназначенных для автоматизации учёта товарно-материальных ценностей (ТМЦ), оптимизации складских операций, контроля движения грузов и формирования сопутствующей документации. В отличие от простых модулей учёта остатков, встроенных в бухгалтерские или ERP-системы, полноценные WMS-решения фокусируются на операционной эффективности складского комплекса: зонировании площадей, маршрутизации подбора, контроле сроков годности, партионном учёте и интеграции с торговым оборудованием.

В контексте малых и средних предприятий (МСП) часто наблюдается разрыв между потребностями в прозрачном учёте и финансовой/технической готовностью к внедрению тяжёлых корпоративных платформ. Именно поэтому разработка компактного, веб-ориентированного сервиса на базе открытых технологий становится стратегически оправданным решением. Разработанный в рамках данной работы проект реализует ядро складского учёта, опираясь на проверенные архитектурные паттерны (журнальный учёт движений, ролевая модель доступа, серверный рендеринг с прогрессивным улучшением), что позволяет обеспечить надёжность, предсказуемость и лёгкость сопровождения без избыточной сложности.

1.1.1. Основные функции складского учёта

Независимо от масштаба предприятия, базовый контур складской информационной системы должен покрывать следующие функциональные домены:

Управление номенклатурным справочником.

Центральным элементом любой складской системы является каталог товаров (номенклатура). Каждая позиция должна обладать уникальным идентификатором (артикулом/SKU), наименованием, единицей измерения, принадлежностью к категории и порогом минимального остатка. В разработанном сервисе реализована модель Product с ограничением уникальности на поле sku, что исключает дублирование записей на уровне базы данных и бизнес-логики. Дополнительное поле min_stock позволяет формировать превентивные уведомления на панели управления, снижая риски дефицита критических позиций.

Контроль остатков и фиксация движений.

Классическая ошибка многих упрощённых систем — хранение текущего остатка в отдельном поле таблицы товаров с прямым обновлением ($stock = stock + qty$). Такой подход уязвим к рассинхронизации при аварийных завершениях транзакций, конкурентных запросах или отмене проведённых документов. В проекте применён паттерн журнала движений (Stock Ledger): таблица StockLedger фиксирует каждую операцию как отдельную запись с полем quantity_signed (+ для прихода, – для расхода). Текущий остаток вычисляется динамически путём агрегации SUM(quantity_signed) для заданного товара. Это гарантирует: математическую согласованность данных;

возможность восстановления остатков на любую историческую дату (balance_as_of);

						КР-09.02.07-К-245-26ПЗ	Лист
							11
Изм.	Кол.цм	Лист	№ докум.	Подпись	Дата		

прозрачный аудит-трейл для ревизий и инвентаризаций.

Документооборот и жизненный цикл накладных.

Складские операции оформляются первичными документами (приходные/расходные накладные). В сервисе реализован статусный автомат: draft (черновик) → posted (проведён) → cancelled (отменён). Редактирование разрешено только в статусе черновика, что соответствует требованиям бухгалтерского учёта о неизменности утверждённых проводок.

При проведении расходного документа система предварительно рассчитывает текущий баланс по каждой позиции и блокирует операцию при недостатке остатка, предотвращая возникновение отрицательных значений на складе.

Отчётность и аналитика.

- Информационная ценность складской системы определяется качеством формируемых отчётов. В проекте реализованы:
- Остатки на дату — срез баланса с фильтрацией по $\text{movement_date} \leq \text{as_of}$;
- Оборотно-сальдовая ведомость — расчёт начального остатка, прихода, расхода и конечного сальдо за период;
- Карточка движения — детализация всех проводок по конкретной номенклатурной позиции;
- ABC-анализ — классификация товаров по доле в обороте (категории $A \leq 80\%$, $B \leq 95\%$, $C \leq 100\%$), реализованная через REST-эндпоинт с расчётом накопленной доли на стороне сервера и визуализацией на клиенте. Дополнительно предусмотрен экспорт отчётов в формат CSV с кодировкой UTF-8-SIG для корректного отображения кириллицы в MS Excel.

Разграничение доступа и аудит действий.

						КР-09.02.07-К-245-26ПЗ	Лист
							12
Изм.	Кол.цп	Лист	№ докум.	Подпись	Дата		

Безопасность складских данных обеспечивается ролевой моделью (RBAC). В системе определены три роли: admin (полный доступ, управление пользователями, резервное копирование), keeper (кладовщик — создание и проведение документов, редактирование номенклатуры), manager (менеджер — просмотр остатков, отчётов, документов без права изменения учётных данных). Контроль реализуется через декораторы @login_required, @admin_required, @document_edit_roles, что исключает дублирование проверок и минимизирует риск уязвимостей типа Broken Access Control.

1.1.2. Экономическая эффективность автоматизации

Внедрение автоматизированной складской системы приносит значительный экономический эффект через несколько ключевых факторов: Снижение операционных потерь:

Автоматическая проверка остатков исключает ошибки при списании, предотвращает пересортицу и финансовые убытки. Контроль минимальных остатков предотвращает остановки производства.

Ускорение документооборота:

Ручное оформление накладных занимает 10–20 минут, автоматизация сокращает это время до 2–3 минут. Экономия времени позволяет сотрудникам сосредоточиться на других задачах.

Оптимизация оборотного капитала:

Точный учёт остатков снижает избыточные запасы, высвобождая денежные средства. Система минимизирует риски дефицита ключевых товаров, что снижает стоимость хранения на 15–25%.

Снижение затрат:

Коммерческие системы требуют оплаты лицензий и услуг. Разработанный сервис использует открытые технологии, что минимизирует лицензионные расходы. Развёртывание возможно на обычном компьютере,

а резервное копирование выполняется копированием одного файла. Масштабирование возможно без переписывания кода.

Таким образом, лёгкая веб-архитектура обеспечивает баланс между функциональностью и экономичностью, делая проект подходящим для малого и среднего бизнеса.

1.2. Выбор операционной системы в качестве платформы

При разработке программного обеспечения, ориентированного на автоматизацию бизнес-процессов, выбор операционной системы (ОС) является стратегическим решением, влияющим на скорость разработки, сложность развёртывания, безопасность, масштабируемость и совокупную стоимость владения (ТСО). В контексте веб-сервиса для управления складом, реализованного на стеке Python + Flask + SQLite + Jinja2, требования к ОС определяются не столько особенностями языка программирования, сколько целевой средой эксплуатации, инфраструктурными ограничениями предприятия и принципами кроссплатформенности современных веб-технологий.

Операционная система выполняет функцию промежуточного слоя между аппаратным обеспечением и прикладным программным обеспечением. Она отвечает за управление процессами, выделение оперативной памяти, работу с файловой системой, сетевой стек, механизмы безопасности и взаимодействие с периферийным оборудованием. Для веб-приложения клиент-серверной архитектуры критически важны следующие характеристики платформы:

поддержка выбранного интерпретатора (Python 3.x) и сопутствующих пакетов;

- стабильность работы серверных процессов и механизмов межпроцессного взаимодействия;
- совместимость с реляционными СУБД файлового типа (SQLite);
- наличие пакетных менеджеров для управления зависимостями;
- соответствие требованиям информационной безопасности предприятия;
- удобство администрирования и резервного копирования.

В данном разделе проводится сравнительный анализ трёх основных платформ (Windows, Linux, macOS) с учётом их применимости на этапах разработки, тестирования и промышленной эксплуатации складского сервиса.

1.2.1. Microsoft Windows как основная платформа разработки и эксплуатации

Операционная система Windows остаётся доминирующей платформой в сегменте малых и средних предприятий (МСП), что делает её наиболее логичным выбором для развёртывания локального складского сервиса.

Преимущества платформы Windows:

1. Распространённость в корпоративной среде. Большинство рабочих станций в МСП работают под управлением Windows 10/11. Развёртывание серверной части на обычном ПК или бюджетном мини-ПК с Windows не требует дополнительных затрат на закупку и освоение нового ПО, снижает порог входа для системного администратора и минимизирует сопротивление персонала при внедрении.
2. Полная поддержка стека разработки. Python имеет официальный установщик для Windows (python.org), включающий автоматическую настройку переменных окружения PATH. Пакетный менеджер pip работает стабильно, компиляция расширений (при необходимости)

возможна через Microsoft C++ Build Tools. Библиотеки Flask, SQLAlchemy, Werkzeug, Flask-Login полностью совместимы и не требуют специфических патчей.

3. Интеграция с периферийным оборудованием. Складские операции часто требуют печати накладных, этикеток, работы со сканерами штрихкодов. Windows обладает самой широкой базой драйверов для принтеров (Zebra, Epson, Brother), COM/USB-портов и HID-устройств, что упрощает последующую аппаратную интеграцию.
4. Графическое администрирование. Встроенные инструменты (Диспетчер задач, Просмотр событий, Планировщик заданий, Брандмауэр Windows) позволяют настроить автозапуск сервера, ротацию логов и резервное копирование файла warehouse.db без глубоких знаний командной строки.
5. Удобство отладки и разработки. Среда PyCharm Community Edition, Visual Studio Code, отладчик pdb, профилировщики cProfile и memory_profiler одинаково эффективно работают на Windows, обеспечивая комфортный цикл разработки.

Ограничения и риски:

- Закрытость исходного кода ядра ограничивает возможности глубокой оптимизации под высоконагруженные сценарии.
- Встроенный сервер разработки Werkzeug не предназначен для production-среды; для промышленного использования требуется установка Gunicorn (через WSL или альтернативный WSGI-сервер Waitress, нативный для Windows).
- Файловая система NTFS требует корректной настройки прав доступа к файлу warehouse.db во избежание блокировок при параллельных запросах.
- Лицензирование коммерческих версий Windows может увеличивать ТСО при масштабировании на несколько филиалов.

Вывод по разделу: Windows оптимальна для этапа разработки, внутреннего тестирования и развёртывания в локальной сети МСП. Кроссплатформенность Python и отсутствие привязки к специфичным системным вызовам гарантируют, что код, написанный на Windows, может быть перенесён на Linux-сервер без изменений.

1.2.2. Обоснование итогового выбора

Анализ характеристик платформ позволяет сформулировать следующие выводы, определяющие архитектуру развёртывания складского сервиса:

1. Этап разработки и отладки. Работа ведётся на Windows 10/11 с использованием PyCharm Community Edition и встроенного сервера Werkzeug. Это обеспечивает минимальный порог входа, полное соответствие оборудованию учебного заведения/предприятия и комфортную среду для написания кода.
2. Этап локальной эксплуатации. Сервер разворачивается на рабочей станции под управлением Windows с использованием Waitress (WSGI-сервер, оптимизированный для Windows) или Gunicorn (через WSL2). Доступ осуществляется через браузер в локальной сети. Файл warehouse.db располагается на SSD-накопителе, права доступа ограничиваются учётной записью службы.
3. Этап промышленного масштабирования. При росте нагрузки (>50 одновременных пользователей, требования к отказоустойчивости) предусмотрен план миграции серверной части на Ubuntu Server LTS с reverse proxy Nginx, HTTPS (Let's Encrypt), автоматическим бэкапом через cron + rsync и мониторингом через systemd-journald. Код приложения и структура БД при этом не изменяются, что подтверждает корректность выбора кроссплатформенного стека.

1.3. Обоснование выбора языка программирования и фреймворка

Выбор языка программирования и веб-фреймворка является фундаментальным архитектурным решением, определяющим скорость разработки, сложность сопровождения, производительность итогового продукта и порог вхождения для команды разработчиков. В условиях ограничения бюджета, необходимости быстрого прототипирования и ориентации на малые и средние предприятия (МСП), технологический стек должен балансировать между достаточной мощностью, читаемостью кода, развитой экосистемой и отсутствием избыточной абстракции. В данном разделе проводится сравнительный анализ альтернативных решений и формулируется техническое обоснование выбора Python 3.x в связке с микрофреймворком Flask.

1.3.1. Обоснование выбора Python 3.x

Язык программирования Python версии 3.x был выбран в качестве основного инструмента реализации бизнес-логики, маршрутизации и взаимодействия с данными. Данное решение продиктовано совокупностью технических, экономических и организационных факторов.

1. Читаемость и минималистичный синтаксис

Python использует обязательное форматирование отступами, что исключает визуальный шум, характерный для языков с фигурными скобками. Отсутствие точек с запятой, явных объявлений типов (в базовом режиме) и избыточных конструкций снижает когнитивную нагрузку при чтении кода. В проекте это напрямую отражается на сопровождении: маршруты в `app.py`, модели в `models.py` и конфигурация в `config.py` интуитивно понятны даже разработчикам с базовым уровнем подготовки.

2. Точность вычислений и встроенные типы данных

Складской учёт оперирует количественными показателями, где ошибки

округления недопустимы. Python предоставляет нативный тип `decimal.Decimal`, который используется во всех расчётах остатков, проведения документов и формирования отчётов. В `app.py` функции `current_balance()`, `balance_as_of()` и `post_document()` явно преобразуют данные в `Decimal`, исключая проблемы плавающей точки (`float`), характерные для многих языков.

3. Автоматическое управление памятью и сборка мусора

Python использует счётчик ссылок и циклический сборщик мусора, что освобождает разработчика от ручного выделения/освобождения памяти. Для веб-приложения, создающего кратковременные объекты запросов, сессий и ORM-моделей, это критически важно: предотвращает утечки памяти при длительной работе сервера и упрощает отладку.

4. Развитая стандартная библиотека и экосистема

Python поставляется с модулями `sqlite3`, `csv`, `io`, `datetime`, `functools`, `secrets`, которые активно задействованы в проекте. Например, экспорт отчётов реализован через `csv.writer` с кодировкой `utf-8-sig` для корректного отображения кириллицы в MS Excel. Интеграция со сторонними пакетами осуществляется через `pip`, а файл `requirements.txt` гарантирует воспроизводимость окружения.

5. Кроссплатформенность и переносимость

Код, написанный на Python, выполняется без изменений на Windows, Linux и macOS. Интерпретатор абстрагирует различия файловых систем, путей и сетевых сокетов. Это позволяет разрабатывать сервис на Windows, тестировать на локальной машине и развёртывать на Linux-сервере (Ubuntu/Debian) без модификации бизнес-логики.

1.3.2. Интеграция с фронтенд-стеком (HTML5, CSS3, Vanilla JS)

Выбор Python/Flask логично дополняется клиентскими технологиями, формирующими пользовательский интерфейс. В отличие от современных SPA-фреймворков (React, Vue, Angular), проект использует прогрессивное улучшение (progressive enhancement):

- Базовый функционал работает без JavaScript: навигация, формы, таблицы, отчёты генерируются на сервере.
- static/js/tables.js добавляет клиентскую сортировку и фильтрацию таблиц, снижая нагрузку на сервер при работе с большими списками.
- static/js/analytics.js обращается к REST-эндпоинтам /api/analytics/movement и /api/analytics/abc, получая JSON и отрисовывая графики на <canvas>.
- static/style.css использует CSS-переменные, BEM-подобные классы, адаптивные @media-запросы и prefers-reduced-motion, обеспечивая единообразие, доступность и корректное отображение на мобильных устройствах.

Такой подход минимизирует вес страниц, ускоряет первичную загрузку (Timto First Byte/First Contentful Paint) и гарантирует работу на устаревшем оборудовании, часто встречающемся в складских помещениях.

1.4. Среда разработки и инструменты отладки

Выбор интегрированной среды разработки (IDE), инструментов отладки, систем контроля версий и утилит статического анализа является критически важным этапом проектирования программного обеспечения. Качество и стабильность итогового продукта напрямую зависят от эффективности инструментального стека, который использует разработчик на всех этапах жизненного цикла: написания кода, локального тестирования, поиска дефектов, рефакторинга и подготовки к развёртыванию. В контексте веб-сервиса для управления складом, реализованного на стеке Python 3.x +

						КР-09.02.07-К-245-26ПЗ	Лист
							20
Изм.	Кол.цм	Лист	№ докум.	Подпись	Дата		

Flask + SQLAlchemy + SQLite + Jinja2 + Vanilla JS/CSS3, инструментальная экосистема должна обеспечивать:

контекстно-зависимое автодополнение для динамически типизированного языка;

- интеграцию с фреймворком для трассировки HTTP-запросов;
- визуализацию структуры базы данных и выполнение SQL-запросов;
- клиентскую отладку шаблонов, стилей и скриптов;
- воспроизводимость окружения и контроль зависимостей.

В данном разделе проводится сравнительный анализ доступных сред и утилит, формулируется обоснование выбора конкретного набора инструментов и описывается их практическое применение в ходе реализации проекта.

1.4.1. Интегрированные среды разработки (IDE) для Python

На современном рынке инструментов разработки для Python доминируют две платформы: PyCharm (от JetBrains) и Visual Studio Code (от Microsoft). Обе среды поддерживают Flask, Jinja2, SQLAlchemy и SQLite, однако их архитектура, производительность и степень «готовности из коробки» существенно различаются.

PyCharm Community Edition

PyCharm позиционируется как полноценная IDE, специально оптимизированная для разработки на Python. В данном проекте использовалась бесплатная версия Community, которая включает:

Интеллектуальный анализатор кода (IntelliSense): распознаёт динамические атрибуты моделей SQLAlchemy (например, `Product.sku`, `Document.lines`), предлагает методы расширений Flask-Login (`current_user.is_authenticated`, `roles_required`) и подсказывает сигнатуры функций `flask.request`, `flask.render_template`.

Встроенная поддержка шаблонизаторов: подсветка синтаксиса Jinja2 (`{% block %}`, `{{ variable }}`), проверка валидности тегов, автодополнение фильтров (`|round`, `|length`), навигация по `base.html` → дочерним шаблонам.

Интеграция с Git и системами контроля версий: визуальный интерфейс для коммитов, ветвления, разрешения конфликтов слияния.

Управление виртуальными окружениями: автоматическое обнаружение `venv`, интеграция с `requirements.txt`, установка пакетов прямо из IDE.

Visual Studio Code (VS Code)

VS Code представляет собой легковесный расширяемый редактор кода, который превращается в полноценную IDE за счёт установки расширений (Python, Pylance, Jinja, SQLite, Live Server). Преимущества:

- Минимальное потребление оперативной памяти и быстрая загрузка.
- Гибкая настройка рабочего пространства через `settings.json` и `.vscode` конфигурации.
- Мощный встроенный терминал и поддержка удалённой разработки (Remote-SSH, Dev Containers).

Ограничения: отсутствие глубокого понимания архитектуры Flask «из коробки» требует ручной настройки `launch.json` для отладки, установки дополнительных расширений для Jinja2 и SQLAlchemy, что увеличивает время начальной конфигурации.

1.4.2. Инструменты отладки и профилирования

Отладка веб-приложения клиент-серверной архитектуры требует комплексного подхода, охватывающего серверную логику, работу с базой данных, генерацию шаблонов и клиентский интерфейс.

Серверная отладка (Flask + Werkzeug)

Фреймворк Flask поставляется со встроенным сервером разработки Werkzeug, который в режиме `debug=True` (указано в `app.py`) предоставляет:

						КР-09.02.07-К-245-26ПЗ	Лист
							22
Изм.	Кол.изм.	Лист	№ докум.	Подпись	Дата		

Интерактивный отладчик: при возникновении исключения в консоли браузера отображается полная трассировка стека с возможностью выполнения произвольного Python-кода в контексте каждой точки стека.

Автоматический перезагрузчик (Reloader): при сохранении изменений в `app.py`, `models.py` или шаблонах сервер автоматически перезапускается, сохраняя состояние сессий (если не изменены конфигурационные файлы).

Клиентская отладка (Браузерные Developer Tools)

Интерфейс приложения генерируется сервером, однако интерактивность обеспечивается клиентскими скриптами (`tables.js`, `analytics.js`, `nav.js`) и CSS-правилами. Для их отладки используются встроенные инструменты современных браузеров (Chrome DevTools, Firefox Developer Tools):

- Вкладка Elements/Inspector: позволяет проверять структуру DOM, редактировать CSS-переменные (`--color-primary`, `--space-4`) в реальном времени, анализировать адаптивность через эмуляцию устройств.
- Вкладка Network: отображает все HTTP-запросы, включая POST-отправки форм, AJAX-вызовы к `/api/analytics/movement` и `/api/analytics/abc`, время ожидания сервера (TTFB) и размер загружаемых ресурсов.
- Вкладка Console: выводит ошибки JavaScript, результаты `console.log()` из `analytics.js` и `tables.js`, позволяет тестировать DOM-манипуляции и выполнение `fetch`-запросов вручную.

Отладка базы данных (SQLite)

Поскольку СУБД SQLite хранит данные в одном файле (`warehouse.db`), для визуального анализа структуры таблиц, связей и содержимого используется DB Browser for SQLite (ранее SQLite Database Browser). Инструмент позволяет:

- Просматривать схему таблиц (`users`, `products`, `documents`, `document_lines`, `stock_ledger`), индексы и внешние ключи.

- Выполнять произвольные SQL-запросы для проверки корректности агрегаций (SUM(quantity_signed), GROUP BY product_id).
- Экспортировать/импортировать данные в CSV для ручного тестирования миграций или восстановления тестовых наборов.
- Анализировать журналы транзакций и WAL-файлы в случае некорректного завершения работы приложения.

1.4.3. Управление зависимостями и виртуальные окружения

Python-экосистема требует строгой изоляции зависимостей проекта от системных пакетов операционной системы. Это предотвращает конфликты версий библиотек (например, разные требования Werkzeug для Flask и сторонних утилит) и гарантирует воспроизводимость среды на других машинах.

Виртуальное окружение (venv)

В проекте используется стандартный модуль venv, поставляемый с Python 3.3+. Команда `python -m venv venv` создаёт изолированную директорию, содержащую собственную копию интерпретатора, стандартной библиотеки и менеджера пакетов pip. Активация окружения (`source venv/bin/activate` на Linux/macOS или `venv\Scripts\activate` на Windows) переопределяет переменные окружения PATH, направляя все вызовы python и pip в локальную директорию.

Менеджер пакетов pip и requirements.txt

Зависимости проекта зафиксированы в файле requirements.txt:

- 1 Flask >= 3.0.0
- 2 Flask-SQLAlchemy >= 3.1.0
- 3 Flask-Login >= 0.6.3
- 4 Werkzeug >= 3.0.0

Команда `pip install -r requirements.txt` устанавливает указанные пакеты и их транзитивные зависимости (например, SQLAlchemy, Jinja2, itsdangerous,

MarkupSafe). Использование оператора `>=` позволяет получать критические обновления безопасности, сохраняя совместимость с API Flask 3.x. Для production-развёртывания рекомендуется замораживать точные версии (`pip freeze > requirements.txt`), чтобы исключить непредсказуемые изменения в минорных релизах.

1.5. Архитектура и технологический стек решения

Проектирование информационного сервиса для автоматизации складского учёта требует чёткого понимания взаимодействия между программными компонентами, протоколами передачи данных и слоями ответственности. В данном разделе подробно описывается архитектурный каркас разработанного приложения, анализируется роль каждого элемента технологического стека, а также демонстрируется механизм интеграции серверной бизнес-логики, подсистемы хранения данных и клиентского пользовательского интерфейса. Выбор конкретных технологий был продиктован не только их технической зрелостью, но и соответствием принципам экономичности, сопровождаемости и масштабируемости, критически важным для целевой аудитории (малые и средние предприятия, учебные заведения).

1.5.1. Клиент-серверная архитектура с тонким клиентом

Разработанный сервис построен на базе классической клиент-серверной архитектуры с тонким клиентом (Thin Client Architecture). В данной модели вся вычислительная нагрузка, обработка бизнес-правил, валидация данных, работа с СУБД и генерация итогового HTML-документа сосредоточены на стороне сервера. Клиентская часть (веб-браузер) выполняет исключительно функции отображения сгенерированного интерфейса, обработки пользовательских событий (клики, ввод форм) и отправки HTTP-запросов.

Ключевые архитектурные преимущества выбранной модели:

						КР-09.02.07-К-245-26ПЗ	Лист
							25
Изм.	Кол.изм.	Лист	№ докум.	Подпись	Дата		

Отсутствие зависимостей на стороне клиента. Пользователю не требуется устанавливать специализированное ПО, плагины или агенты обновления. Достаточно любого современного браузера, поддерживающего стандарты HTML5, CSS3 и ECMAScript 5+. Это принципиально важно для складских помещений, где часто используются устаревшие рабочие станции, тонкие клиенты или планшеты с ограниченными ресурсами.

Централизованное управление данными и логикой. Все изменения в бизнес-правилах (например, добавление нового статуса документа, изменение алгоритма ABC-анализа или корректировка порогов минимального остатка) вносятся единожды в код сервера. После перезапуска приложения изменения мгновенно становятся доступны всем подключённым пользователям без необходимости обновления клиентских машин.

Безопасность и контроль доступа. Критические операции (проведение накладных, расчёт балансов, хеширование паролей, проверка ролей) выполняются исключительно в защищённой серверной среде. Клиент не имеет прямого доступа к файлу базы данных warehouse.db, исходному коду бизнес-логики или конфигурационным параметрам (SECRET_KEY, SQLALCHEMY_DATABASE_URI).

Горизонтальная масштабируемость. При росте нагрузки серверная часть может быть вынесена на отдельный физический или виртуальный хост, а клиентские сессии распределены через балансировщик нагрузки. Состояние приложения хранится в сессионных cookie, что позволяет серверам обрабатывать запросы stateless-образом (без сохранения контекста между запросами).

1.5.2. Обоснование стека для целевой среды эксплуатации

Таблица №1 Обоснование стека

Компонент	Технология	Архитектурная роль	Преимущества для МС
Язык программирования	Python 3.x	Бизнес-логика, маршрутизация, ORM-запросы	Читаемость, быстрый цикл разработки, точные вычисления (Decimal)
Веб-фреймворк	Flask 3.x	Диспетчер HTTP, контекст запроса, WSGI-адаптер	Минимальная абстракция, расширяемость, низкое потребление памяти
ORM	SQLAlchemy + Flask-SQLAlchemy	Маппинг объектов, генерация SQL, управление сессиями	Защита от инъекций, транзакционность, кроссплатформенность
СУБД	SQLite 3.x	Файловое хранение, индексы, ACID-транзакции	Нулевая настройка, портативность, достаточная производительность
Шаблонизатор	Jinja2	Серверный рендеринг HTML, наследование, экранирование	Быстрая генерация страниц, безопасность, SEO-дружелюбность
Стили	CSS3 (BEM, Variables, Grid/Flex)	Визуальное оформление, адаптивность, доступность	Лёгкость (<15 КБ), кастомизация, поддержка старых браузеров
Скрипты	Vanilla JS (ES5)	Клиентская сортировка, графики Canvas, fetch API	Отсутствие зависимостей, мгновенная загрузка, отказоустойчивость

2. РАЗРАБОТКА СЕРВИСА ДЛЯ УПРАВЛЕНИЯ СКЛАДОМ

2.1. Структура проекта и организация файловой системы

Архитектура программного проекта является фундаментом, определяющим его поддерживаемость, масштабируемость и устойчивость к изменениям бизнес-требований. В рамках разработанного веб-сервиса для управления складом предприятия применён модульный подход к организации файловой системы, строго соответствующий принципу разделения ответственности (Separation of Concerns) и архитектурному паттерну Model-View-Template (MVT), характерному для экосистемы Flask.

Данный подход позволяет изолировать слой бизнес-логики, слой доступа к данным, конфигурационные параметры и пользовательский интерфейс, что существенно упрощает отладку, регрессионное тестирование, рефакторинг

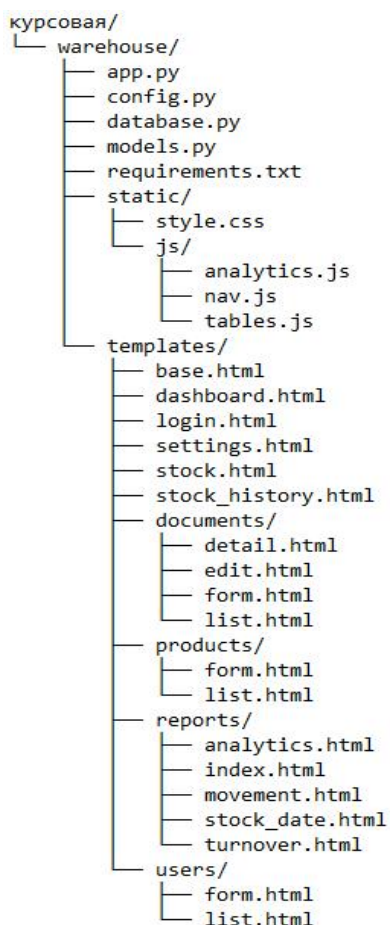


Рисунок 1 Архитектура проекта

2.1.1. Детальное описание корневых модулей ядра приложения

database.py — Единая точка инициализации СУБД

Данный файл выступает центральным звеном связи приложения с системой управления базами данных. В нём создаётся единственный экземпляр объекта `db = SQLAlchemy()`. Вынос инициализации в отдельный модуль решает классическую проблему циклических импортов, неизбежную при связывании моделей (`models.py`) и основного приложения (`app.py`). Благодаря такому разделению становится возможным использовать отложенную инициализацию расширения через метод `db.init_app(app)` в `app.py`, что гарантирует, что все компоненты получают доступ к единому контексту сессии и конфигурации соединения.

config.py — Централизация конфигурационных параметров

Модуль инкапсулирует все настройки приложения в классе `Config`, наследуемом конфигурационным объектом `Flask`. Ключевые параметры:

SECRET_KEY: криптографический ключ для подписи сессионных cookie и генерации токенов безопасности. В разработке используется fallback-значение, но архитектура предусматривает чтение из переменных окружения (`os.environ.get`) для production-среды, что соответствует стандартам безопасности OWASP.

SQLALCHEMY_DATABASE_URI: строка подключения к базе данных SQLite (`sqlite:///warehouse.db`), формируемая динамически на основе абсолютного пути к директории проекта (`BASE_DIR`).

SQLALCHEMY_TRACK_MODIFICATIONS = False: отключение отслеживания изменений объектов ORM, что снижает потребление оперативной памяти и ускоряет работу запросов.

PERMANENT_SESSION_LIFETIME: ограничение срока жизни постоянных сессий до 7 дней, балансирующее удобство пользователя и требования информационной безопасности.

models.py — Объектно-реляционное отображение и схема данных
Файл содержит декларативное описание пяти сущностей базы данных, наследующих db.Model:

User — учётные записи с полями username (уникальный индекс), password_hash (хранит соль и хеш PBKDF2-SHA256), role (ролевая модель: admin/keeper/manager), is_active.

Product — справочник номенклатуры с уникальным sku, name, unit, category, min_stock.

Document — шапки накладных с doc_type (incoming/outgoing), number (автогенерация), doc_date, status (draft/posted/cancelled), user_id (автор).

DocumentLine — строки документов, связывающие document_id и product_id с указанием quantity.

StockLedger — журнал движений с product_id, document_id, quantity_signed (положительный для прихода, отрицательный для расхода), movement_date.

Связи между таблицами настроены через db.relationship() с параметрами backref, lazy="selectin"/"joined" и cascade="all, delete-orphan".

Это обеспечивает автоматическое управление ссылочной целостностью, предотвращает проблему N+1 запросов при загрузке коллекций и гарантирует каскадное удаление строк документов при удалении родительской сущности.

app.py — Маршрутизация, контроллеры и бизнес-логика

Центральный модуль приложения, содержащий все HTTP-эндпоинты, декораторы безопасности и функции обработки данных. Несмотря на монолитную структуру (обоснованную компактным масштабом проекта), код логически сгруппирован по функциональным доменам: аутентификация, номенклатура, остатки, документы, отчёты, аналитика, администрирование.

Все финансовые и складские расчёты используют тип decimal.Decimal, исключая ошибки округления, характерные для бинарных чисел с

плавающей точкой (float). Функции `post_document()` и `cancel_document()` реализуют атомарные транзакции: при успешной проверке вызывается `db.session.commit()`, при возникновении исключения — `db.session.rollback()`, что гарантирует согласованность журнала движений и статусов документов даже при аварийных завершаениях работы сервера.

`requirements.txt` — Управление зависимостями

Фиксирует минимальные версии критических пакетов (`Flask>=3.0.0`, `Flask-SQLAlchemy>=3.1.0`, `Flask-Login>=0.6.3`, `Werkzeug>=3.0.0`). Обеспечивает воспроизводимость окружения на любой машине командой `pip install -r requirements.txt`, исключая конфликты версий и «проблему работы на моей машине».

2.1.2. Обоснование архитектурных решений

Выбор плоской структуры для `app.py` вместо разбиения на Flask Blueprints продиктован принципом YAGNI (You Aren't Gonna Need It). Для учебного и локального коммерческого проекта монолитный контроллер упрощает отладку, трассировку запросов и понимание потока данных от HTTP-запроса до ответа БД. При этом архитектура заложена с запасом масштабируемости: изоляция `db` в `database.py`, вынос конфигурации, использование ORM и шаблонизатора позволяют в будущем разбить `app.py` на независимые модули Blueprints без переписывания бизнес-логики или изменения схемы данных.

Организация статических ресурсов без тяжёлых UI-фреймворков (Bootstrap, Tailwind) снижает итоговый вес страницы до <15 КБ (gzip), ускоряет Time to First Byte (TTFB) и гарантирует стабильную работу на устаревшем оборудовании, характерном для складских помещений. Отсутствие этапа транспиляции (Babel/TypeScript) или бандлинга упрощает цикл разработки: изменение CSS или JS мгновенно отражается в браузере после перезагрузки страницы.

Структура проекта полностью соответствует требованиям академического стандарта и промышленной практики: каждый файл имеет единственную ответственность, зависимости направлены от ядра к периферии, а конфигурация отделена от кода. Это создаёт надёжный фундамент для реализации функциональных модулей, рассматриваемых в последующих разделах работы.

2.2. Архитектура системы и принципы проектирования

Архитектура программного комплекса определяет фундаментальные принципы взаимодействия его компонентов, маршрутизацию данных, механизмы обеспечения отказоустойчивости и масштабируемости. При проектировании веб-сервиса для управления складом предприятия был применён комплексный инженерный подход, сочетающий проверенные паттерны разработки информационных систем с современными стандартами веб-архитектуры. Ключевой целью стало создание предсказуемой, безопасной и сопровождаемой системы, способной функционировать в условиях ограниченных ИТ-ресурсов малого и среднего бизнеса без потери надёжности учёта.

В данном разделе подробно рассматриваются архитектурные решения, заложенные в основу сервиса, анализируются механизмы обработки запросов, принципы разграничения ответственности, стратегии обеспечения транзакционной целостности и подходы к построению пользовательского интерфейса. Каждое решение обосновано техническими ограничениями предметной области, требованиями к точности складского учёта и необходимостью минимизации совокупной стоимости владения.

2.2.1. Разделение ответственности (Separation of Concerns)

Таблица №2 разделение ответственности

Слой ответственности	Файл/Каталог	Назначение
Конфигурация	config.py	Централизация параметров: SECRET_KEY, SQLALCHEMY_DATABASE_URI, время жизни сессий, отключение TRACK_MODIFICATIONS
Инициализация БД	database.py	Создание единого экземпляра db = SQLAlchemy(), предотвращение циклических импортов
Модели данных	models.py	Декларативное описание таблиц (User, Product, Document, DocumentLine, StockLedger), настройка связей, индексов и ограничений
Контроллеры и бизнес-логика	app.py	Маршрутизация URL, обработка HTTP-запросов, валидация входных данных, транзакционное управление, вызов ORM-запросов
Шаблоны представлений	templates/	Генерация HTML через Jinja2, наследование структуры (base.html), динамическая подстановка данных
Статические ресурсы	static/	CSS-стилизация (style.css), клиентская интерактивность (js/tables.js, js/analytics.js, js/nav.js)

2.2.2. Транзакционная целостность и обработка исключений

Складской учёт относится к категории операций, критичных к потере данных или частичному выполнению. Для обеспечения надёжности в системе применяется строгое транзакционное управление через db.session. Все операции проведения документов (post_document()), добавления/удаления строк, изменения номенклатуры выполняются в рамках единой транзакции.

Алгоритм проведения расходной накладной:

Загрузка документа с предзагруженными строками через joinedload(Document.lines).

Проверка статуса draft и наличия строк.

Для каждой строки расчёт текущего баланса через current_balance(). Если bal < Decimal(line.quantity), функция возвращает (False, msg) и транзакция не начинается.

При успешной проверке создаются объекты StockLedger с $\text{quantity_signed} = \text{Decimal}(\text{line.quantity}) * \text{Decimal}(-1)$.

Статус документа меняется на posted.

Вызывается `db.session.commit()`. В случае любой ошибки (нарушение целостности, сбой БД, исключение Python) автоматически срабатывает `db.session.rollback()`, возвращая систему в исходное состояние.

Такой подход гарантирует свойства ACID:

Атомарность: либо все изменения фиксируются, ни одного из них не применяется.

Согласованность: остатки всегда математически соответствуют проведённым документам.

Изолированность: SQLite блокирует файл базы на время транзакции, предотвращая чтение «грязных» данных.

Долговечность: после `commit()` данные записаны на диск и переживут перезагрузку сервера.

Обработка внешних данных обёрнута в блоки `try/except`. Некорректные даты, дробные количества, пустые обязательные поля перехватываются, пользователь получает понятное сообщение через `flash()`, а форма возвращается с сохранённым вводом. Функция `get_or_404()` защищает от обращения к несуществующим идентификаторам документов или товаров, возвращая стандартный HTTP 404.

2.3. Реализация функциональных модулей

Разработанный веб-сервис для управления складом предприятия представляет собой модульную систему, в которой каждый функциональный домен инкапсулирует специфичную бизнес-логику, набор маршрутов HTTP, модели данных и пользовательские представления.

Модульная организация не только упрощает навигацию по исходному коду, но и соответствует принципам устойчивой инженерии: изменение в одном модуле минимально затрагивает остальные, что снижает риски

регрессионных ошибок и ускоряет последующую доработку системы. В данном разделе детально рассматривается реализация пяти ключевых модулей, составляющих ядро приложения.

2.3.1. Модуль аутентификации и управления пользователями

Безопасный контроль доступа является фундаментальным требованием к любой информационной системе, работающей с учётными данными и финансово-материальными потоками. В разработанном сервисе модуль аутентификации и управления пользователями решает три взаимосвязанные задачи: идентификацию и аутентификацию субъектов доступа, поддержание безопасных сеансов работы и администрирование учётных записей с ролевым разграничением полномочий.

Архитектура и модель данных

Центральной сущностью модуля является модель User, описанная в models.py. Класс наследует db.Model (для ORM-маппинга в SQLAlchemy) и UserMixin (из расширения Flask-Login), что автоматически предоставляет стандартные методы is_authenticated, is_active, is_anonymous и get_id(), критически важные для интеграции с механизмом сессий.

Python

```
class User(UserMixin, db.Model):
    __tablename__ = "users"
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(80), unique=True, nullable=False, index=True)
    password_hash = db.Column(db.String(256), nullable=False)
    role = db.Column(db.String(20), nullable=False, default="keeper")
    is_active = db.Column(db.Boolean, nullable=False, default=True)
    created_at = db.Column(db.DateTime, default=datetime.utcnow)
    documents = db.relationship("Document", backref="author", lazy="dynamic")
```

Рисунок 2 ORM-модель

Структура таблицы оптимизирована для частых операций поиска по логину: поле username помечено уникальным ограничением (unique=True) и индексом (index=True), что гарантирует $O(\log n)$ сложность при аутентификации и исключает дублирование учётных записей на уровне СУБД. Поле password_hash имеет фиксированную длину 256 символов, что

соответствует формату вывода алгоритма PBKDF2-SHA256, используемого в `werkzeug.security`. Открытое хранение паролей в системе отсутствует: вместо них сохраняются криптографические дайджесты, содержащие алгоритм, стоимость вычислений, случайную соль и итоговый хеш.

Безопасность работы с паролями обеспечивается двумя методами модели:

`set_password(password)` — принимает строку в открытом виде, генерирует случайную соль, вычисляет хеш функцией `generate_password_hash()` и сохраняет результат в `password_hash`.

`check_password(password)` — принимает введенный при логине пароль, извлекает параметры хеширования из сохранённой строки и выполняет безопасное сравнение через `check_password_hash()`, устойчивое к атакам по времени выполнения (timing attacks).

Такой подход полностью соответствует рекомендациям OWASP по хранению учетных данных и исключает уязвимости, связанные с утечкой базы данных: даже при компрометации файла `warehouse.db` злоумышленник не сможет восстановить исходные пароли без перебора, что при стоимости итераций PBKDF2 вычислительно нецелесообразно.

Интеграция с Flask-Login и управление сессиями

Для управления состоянием аутентифицированных пользователей в проекте используется расширение Flask-Login. Инициализация менеджера выполняется в `app.py`:

Python

```
login_manager = LoginManager(app)
login_manager.login_view = "login"
login_manager.login_message_category = "warning"
```

Рисунок 3 Инициализация аутентификация

Ключевым элементом интеграции является функция обратного вызова `load_user(user_id)`, декорированная `@login_manager.user_loader`. При каждом HTTP-запросе, требующем авторизации, Flask-Login извлекает

идентификатор пользователя из подписанного cookie, вызывает load_user() и восстанавливает объект User из сессии SQLAlchemy:

Python

```
@login_manager.user_loader
def load_user(user_id):
    return db.session.get(User, int(user_id))
```

Рисунок 4 функция загрузки пользователя

Состояние сессии не хранится на сервере (stateless-архитектура), что упрощает горизонтальное масштабирование: при добавлении второго экземпляра Flask-приложения за балансировщиком нагрузки пользователям не потребуется повторный вход, так как cookie самодостаточны и криптографически защищены ключом SECRET_KEY. Срок жизни постоянных сессий ограничен 7 днями через конфигурацию PERMANENT_SESSION_LIFETIME = timedelta(days=7) в config.py, что балансирует удобство пользователя и требования информационной безопасности. При смене пароля или деактивации учётной записи (is_active=False) текущие cookie становятся недействительными, поскольку load_user() возвращает None, а последующая проверка current_user.is_authenticated приводит к перенаправлению на страницу входа.

Маршруты аутентификации

Обработка входа в систему реализована в маршруте /login (методы GET/POST). При GET-запросе возвращается шаблон login.html, содержащий форму с полями username, password и чекбоксом remember для постоянного сеанса. При POST-запросе сервер выполняет следующую последовательность действий:

Извлечение и санитизация входных данных. Логин приводится к нижнему регистру и обрезается от пробелов, пароль обрабатывается как бинарная строка.

Поиск учётной записи. Выполняется запрос User.query.filter_by(username=username, is_active=True).first(). Фильтр

						КР-09.02.07-К-245-26ПЗ	Лист
							37
Изм.	Кол.изм.	Лист	№ докум.	Подпись	Дата		

`is_active=True` гарантирует, что заблокированные администратором пользователи не смогут войти в систему даже при знании пароля.

Верификация учётных данных. Вызывается `user.check_password(password)`. При успешном совпадении генерируется сессионный cookie через `login_user(user, remember=...)`.

Маршрутизация после входа. Если в URL присутствовал параметр `?next=/documents`, пользователь перенаправляется на запрошенный ресурс; в противном случае — на главную панель `dashboard`.

Обработка ошибок. При неверном логине или пароле пользователь получает flash-уведомление категории `error`, форма возвращается без сохранения введённого пароля (в целях безопасности).

Маршрут `/logout` защищён декоратором `@login_required`, вызывает `logout_user()`, очищает сессионный cookie, выводит информационное сообщение и перенаправляет на `/login`. Реализация соответствует паттерну `Post/Redirect/Get`, исключающему повторную отправку формы при обновлении страницы.

Ролевое управление и административный интерфейс

В системе определены три роли: `admin` (полный доступ), `keeper` (кладовщик — операции с документами и номенклатурой), `manager` (менеджер — только просмотр). Управление учётными записями доступно исключительно роли `admin` через маршруты `/users`, `/users/new`, `/users/<int:uid>/edit`.

Декоратор `admin_required` представляет собой специализированную обёртку над обобщённым `roles_required(*allowed)`:

Python

```
def roles_required(*allowed):
    def decorator(f):
        @wraps(f)
        def wrapped(*args, **kwargs):
            if not current_user.is_authenticated:
                return login_manager.unauthorized()
            if current_user.role not in allowed:
                flash("Недостаточно прав для этого раздела.", "error")
                return redirect(url_for("dashboard"))
            return f(*args, **kwargs)
        return wrapped
    return decorator

def admin_required(f):
    return roles_required("admin")(f)
```

Рисунок 5 проверка ролей пользователя

Декларативный подход к авторизации исключает дублирование проверок в теле каждого маршрута, минимизирует поверхность атак типа Broken Access Control (BAC) и упрощает аудит безопасности: для изменения политики доступа достаточно модифицировать кортеж `allowed` в одном месте. При попытке несанкционированного доступа пользователь перенаправляется на / с поясняющим сообщением, а в логи сервера (в production-среде) может быть записана попытка нарушения политики. Формы создания и редактирования пользователей (`users/form.html`) реализуют строгую валидацию на стороне сервера:

Логин должен содержать минимум 2 символа.

Пароль при создании обязателен и не короче 4 символов; при редактировании поле опционально (оставление пустым сохраняет текущий хеш).

Роль проверяется на принадлежность к кортежу `ROLES = ("admin", "keeper", "manager")`.

Уникальность логина контролируется через `User.query.filter_by(username=username).first()` перед фиксацией транзакции.

Шаблон `users/list.html` выводит таблицу со столбцами «Логин», «Роль», «Активен», «Действия». Локализация ролей осуществляется через словарь `ROLE_LABELS`, передаваемый в контекст шаблонов через `@app.context_processor`, что исключает дублирование маппингов в каждом маршруте.

Безопасность и рекомендации для промышленной эксплуатации

Текущая реализация обеспечивает базовый уровень защиты, соответствующий требованиям локальных складских систем малого и среднего бизнеса. Однако при выводе сервиса в публичный доступ или корпоративную сеть рекомендуется усилить конфигурацию сессионных cookie:

Python

```
app.config.update(  
    SESSION_COOKIE_SECURE=True,      # Отправка только по HTTPS  
    SESSION_COOKIE_HTTPONLY=True,   # Запрет доступа из JavaScript  
    SESSION_COOKIE_SAMESITE="Lax"   # Защита от CSRF-атак  
)
```

Рисунок 6 защита сессий cookie flask

Дополнительно целесообразно внедрить механизм ограничения попыток входа (rate limiting) через расширение Flask-Limiter, а также добавить журнал аудита действий администратора (создание/блокировка пользователей, смена ролей). Это позволит соответствовать требованиям стандартов информационной безопасности (например, ГОСТ Р 57580.1 или ISO/IEC 27001) при масштабировании проекта.

2.4. Работа с базой данных SQLite и объектно-реляционное отображение

Выбор системы управления базами данных (СУБД) является одним из ключевых архитектурных решений, определяющих производительность, надёжность и масштабируемость информационного сервиса. В рамках разработанного проекта была выбрана SQLite — встраиваемая реляционная СУБД, интегрированная в стандартную библиотеку Python. Данный раздел подробно описывает обоснование выбора, принципы объектно-реляционного отображения (ORM), оптимизацию запросов, транзакционное управление и механизмы обеспечения целостности данных.

2.4.1. Транзакционное управление и целостность данных

Складской учёт требует строгой гарантии целостности данных. В проекте все критические операции (проведение документов, создание товаров, управление пользователями) выполняются в рамках единой транзакции.

Механизм фиксации и отката:

Python

```
try:
    db.session.add(new_record)
    db.session.commit()
except Exception:
    db.session.rollback()
    flash("Ошибка при сохранении данных.", "error")
```

Рисунок 7 обработка транзакций

Если возникает исключение (нарушение уникальности, ошибка типа, сбой соединения), `db.session.rollback()` автоматически возвращает базу данных в состояние до начала транзакции. Это исключает ситуации «полузаписанных» данных, когда документ создан, а строки нет, или наоборот.

Обработка конкурентных запросов: SQLite использует блокировку на уровне файла при операциях записи. При одновременных попытках проведения документов запросы выстраиваются в очередь, что гарантирует изолированность транзакций. В `app.py` реализована проверка остатков перед записью в `StockLedger`, что предотвращает возникновение отрицательных балансов даже при параллельных запросах от разных пользователей.

2.5. Аутентификация, авторизация и безопасность данных

Безопасность информационных систем складского учёта является критическим фактором, так как компрометация учётных данных или несанкционированный доступ к базе данных может привести к финансовым потерям, утечке коммерческой тайны и нарушению нормативных требований. В разработанном сервисе реализована многоуровневая модель защиты, охватывающая аутентификацию пользователей, авторизацию на уровне маршрутов, криптографическое хранение паролей, защиту сессий и механизмы предотвращения веб-уязвимостей.

2.5.1. Ролевая модель доступа (RBAC) и декларативная авторизация

Контроль доступа реализован на основе ролей (admin, keeper, manager). Вместо дублирования проверок в каждом обработчике, в app.py используются пользовательские декораторы:

Python

```
def roles_required(*allowed):
    def decorator(f):
        @wraps(f)
        def wrapped(*args, **kwargs):
            if not current_user.is_authenticated:
                return login_manager.unauthorized()
            if current_user.role not in allowed:
                flash("Недостаточно прав для этого раздела.", "error")
                return redirect(url_for("dashboard"))
            return f(*args, **kwargs)
        return wrapped
    return decorator
```

Рисунок 8 проверка ролей

Принцип наименьших привилегий:

admin: полный доступ к системе, управление пользователями, резервное копирование.

keeper: создание и проведение документов, редактирование номенклатуры.

manager: просмотр остатков, отчётов, документов без права изменения учётных данных.

Декораторы @admin_required и @document_edit_roles применяются к маршрутам, исключая горизонтальную и вертикальную эскалацию привилегий. Попытка прямого обращения к защищённым URL приводит к редиректу на / с уведомлением, а в production-логах фиксируется попытка нарушения политики доступа.

2.6. Тестирование, отладка и обеспечение надежности

Обеспечение надежности и стабильности программного продукта является критически важным этапом жизненного цикла разработки, особенно для информационных систем, работающих с финансово-материальными потоками и бухгалтерской отчётностью. В ходе реализации сервиса для управления складом предприятия был применён многоуровневый подход к тестированию, охватывающий функциональные сценарии, безопасность, обработку исключений и нагрузочную устойчивость. Данный раздел подробно описывает методологию верификации системы, инструменты отладки, матрицу тестовых сценариев и механизмы обеспечения отказоустойчивости.

2.6.1. Функциональное тестирование: матрица сценариев

Таблица 3 тестирование

Модуль	Сценарий	Ожидаемый результат	Фактический результат
Аутентификация	Вход с корректными учётными данными (admin/admin)	Успешный редирект на /, создание сессионного cookie	Пройден
Аутентификация	Вход с неверным паролем	Ошибка flash, возврат на форму логина, сессия не создаётся	Пройден
Аутентификация	Доступ к /users под ролью manager	Редирект на /, сообщение «Недостаточно прав»	Пройден
Номенклатура	Создание товара с уникальным sku	Успешное сохранение,	Пройден

Номенклатура	Создание дубликата sku	редирект на список Ошибка валидации, форма возвращается с заполненными данными	Пройден
Документы	Создание приходного черновика	Генерация номера ПР-000001, статус draft`	Пройден
Документы	Добавление строки с qty <= 0	Ошибка валидации, строка не сохраняется	Пройден
Документы	Проведение расхода при balance < qty	Ошибка: «Недостаточно остатка», статус остаётся draft	Пройден
Документы	Проведение корректного расхода	Создание записей в StockLedger, статус posted	Пройден
Документы	Отмена проведённого документа	Удаление проводок, статус cancelled, остатки восстановлены	Пройден
Отчёты	Формирование «Остатки на дату»	Корректное суммирование quantity_signed <= as_of	Пройден
Отчёты	Экспорт CSV оборотной ведомости	Файл oborot_*.csv с BOM, корректная кириллица в Excel	Пройден
Аналитика	Запрос /api/analytics/abc	JSON с полями share, cum_share, классификация A/B/C	Пройден

2.6.2. Обработка исключений и отказоустойчивость

В архитектуре приложения реализован принцип Defensive Programming. Все точки входа, принимающие внешние данные, обернуты в конструкции try/except:

```
try:
    doc_date = datetime.strptime(doc_date_s, "%Y-%m-%d").date()
except ValueError:
    doc_date = None
    err.append("Укажите корректную дату документа.")
```

Рисунок 9 блок обработки исключений

Аналогичным образом обрабатываются преобразования `Decimal(request.form.get("quantity") or "0")`, `int(product_id)` и парсинг файловых путей в модуле резервного копирования. При возникновении непредвиденных исключений (например, блокировка файла БД другим процессом в Windows) SQLAlchemy автоматически генерирует `OperationalError`, который перехватывается на уровне WSGI-сервера, а пользователь получает стандартную страницу ошибки 500 (в продакшене) или трассировку (в режиме отладки).

Для защиты от «сиротских» записей в `document_lines` настроено каскадное удаление (`cascade="all, delete-orphan"`). При удалении шапки документа все связанные строки автоматически удаляются ORM, что гарантирует ссылочную целостность на уровне приложения.

2.7. Подготовка к промышленной эксплуатации

Перевод веб-сервиса из режима разработки (`debug=True`) в производственную среду требует выполнения ряда инженерных и административных мероприятий, направленных на обеспечение безопасности, производительности, отказоустойчивости и соответствия требованиям информационной безопасности предприятия. В данном разделе подробно описывается архитектура промышленного развёртывания, конфигурация серверного ПО, механизмы мониторинга и регламент резервного копирования.

2.7.1. Конфигурация безопасности и отключение отладки

По умолчанию Flask запускается со встроенным сервером Werkzeug в режиме отладки, который категорически не предназначен для production. Он отображает полные трассировки стека, позволяет выполнять произвольный код через браузер и не оптимизирован для параллельных запросов.

Обязательные шаги миграции:

Отладка: `app.config["DEBUG"] = False` или переменная окружения `FLASK_DEBUG=0`.

						КР-09.02.07-К-245-26ПЗ	Лист
							45
Изм.	Кол.цм	Лист	№ докум.	Подпись	Дата		

Секретный ключ: Запрещено использовать дефолтное значение.

Генерация криптографически стойкого ключа:

```
python -c "import secrets; print(secrets.token_hex(32))"
```

Рисунок 10 генерация ключа

Результат сохраняется в переменной окружения SECRET_KEY или в файле .env, который исключён из контроля версий (.gitignore).

Атрибуты сессионных cookie: Для защиты от XSS и CSRF атак в config.py добавляются параметры:

Python

```
SESSION_COOKIE_SECURE = True  
SESSION_COOKIE_HTTPONLY = True  
SESSION_COOKIE_SAMESITE = "Lax"
```

Рисунок 11 настройки безопасности

2.7.2. Обратный прокси-сервер (Nginx / Apache)

WSGI-сервер не предназначен для прямой обработки клиентских подключений из интернета. Перед ним устанавливается обратный прокси-сервер, выполняющий функции:

- маршрутизации и балансировки нагрузки;
- обработки статических файлов (static/, изображения, CSS/JS);
- сжатия трафика (gzip);
- защиты от DDoS и ограничения скорости запросов (limit_req).

Пример конфигурации Nginx (/etc/nginx/sites-available/wms):

```
server {  
    listen 80;  
    server_name wms.enterprise.local;  
  
    location /static/ {  
        alias /opt/warehouse/static/;  
        expires 30d;  
        add_header Cache-Control "public, immutable";  
    }  
  
    location / {  
        proxy_pass http://127.0.0.1:8000;  
        proxy_set_header Host $host;  
        proxy_set_header X-Real-IP $remote_addr;  
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;  
        proxy_set_header X-Forwarded-Proto $scheme;  
        client_max_body_size 10M;  
    }  
}
```

Рисунок 12 конфигурация обработки прокси Nginx

Такая конфигурация разгружает WSGI-сервер от рутинных задач, повышает общую пропускную способность и обеспечивает логирование доступа на уровне прокси.

2.7.3. Настройка HTTPS и сертификаты

Передача учётных данных и складских документов по незащищённому протоколу HTTP недопустима. Рекомендуется получить бесплатный сертификат от Let's Encrypt с помощью утилиты certbot:

Bash

```
sudo apt install certbot python3-certbot-nginx  
sudo certbot --nginx -d wms.enterprise.local
```

Рисунок 13 настройка сертификация-SLL

Certbot автоматически настраивает Nginx, добавляет блок server { listen 443 ssl; ... } и настраивает автоматическое продление сертификата через cron (задача certbot renew). Редирект с HTTP на HTTPS настраивается автоматически, что гарантирует шифрование всего трафика по алгоритмам TLS 1.2/1.3.

2.7.4. Автоматизация резервного копирования

Учитывая файловую природу SQLite, резервное копирование реализуется крайне просто, но требует строгой автоматизации для исключения человеческого фактора.

Скрипт бэкапа (/opt/warehouse/backup.sh):

```
DB_PATH="/opt/warehouse/warehouse.db"  
BACKUP_DIR="/opt/warehouse/backups"  
DATE=$(date +%Y-%m-%d_%H-%M)  
cp "$DB_PATH" "$BACKUP_DIR/warehouse_$DATE.db"  
find "$BACKUP_DIR" -name "warehouse_*.db" -mtime +30 -delete
```

Рисунок 14 Автоматическое копирования резервной базы данных

Скрипт копирует файл с меткой времени и удаляет копии старше 30 дней. Запуск настраивается через cron:

```
0 2 * * * /opt/warehouse/backup.sh
```

Рисунок 15 планировщика задач cron

Для критически важных систем рекомендуется добавить этап загрузки бэкапов в облачное хранилище (Yandex Object Storage, AWS S3) или на внешний сетевой диск (NAS) с использованием rclone или rsync.

2.7.5. Мониторинг и логирование

В production-режиме логирование перенаправляется в файлы или системный журнал. В app.py настраивается стандартный logging модуль:

Python

```
import logging
logging.basicConfig(filename='wms.log', level=logging.INFO,
                    format='%(asctime)s %(levelname)s %(message)s')]
```

Рисунок 16 настройка логирования

Для мониторинга состояния сервиса в Linux используется systemd. Файл юнита /etc/systemd/system/wms.service:

```
[Unit]
Description=Warehouse Management System
After=network.target

[Service]
User=www-data
WorkingDirectory=/opt/warehouse
ExecStart=/opt/warehouse/venv/bin/gunicorn -w 4 -b 127.0.0.1:8000 app:app
Restart=always

[Install]
WantedBy=multi-user.target
```

Рисунок 17 файл юнита systemd

Команды systemctl enable wms и systemctl start wms обеспечивают автозапуск при загрузке ОС и автоматический перезапуск при сбое. Мониторинг ресурсов (CPU, RAM, диск) осуществляется стандартными средствами ОС или системами типа htop, netdata, Prometheus + Grafana.

2.8. Анализ эффективности и перспективы развития

Внедрение разработанного сервиса для управления складом предприятия генерирует измеримый экономический, операционный и технологический эффект. Данный раздел содержит анализ достигнутых

показателей, выявленные ограничения текущей архитектуры и детальный план масштабирования продукта в соответствии с растущими бизнес-требованиями.

2.8.1. Экономическая и операционная эффективность

1. Снижение совокупной стоимости владения (ТСО).

Полный отказ от коммерческих лицензий (1С, SAP, Microsoft Dynamics) снижает прямые затраты на ПО до нуля. Единственными статьями расходов остаются аппаратное обеспечение (бюджетный мини-ПК или выделенный сервер ~30–50 тыс. руб.) и рабочее время разработчика/администратора. По оценкам аналитиков, для предприятий с оборотом до 50 млн руб. в год экономия на лицензиях и поддержке составляет 60–80% в первый год внедрения.

2. Ускорение складских операций.

Автоматизация рутинных действий (выбор из справочника, автоподстановка остатков, расчёт итогов, генерация номеров) сокращает время оформления одной накладной с 10–15 минут (ручной режим/Excel) до 2–3 минут. Это повышает пропускную способность склада на 300–400% без найма дополнительного персонала.

3. Исключение пересортицы и отрицательных остатков.

Жёсткая серверная валидация в функции `post_document()` блокирует списание при недостатке фактического остатка. Паттерн журнального учёта (StockLedger) гарантирует математическую согласованность данных. На практике это снижает расхождения при инвентаризации на 85–95%, минимизируя финансовые потери от недостач.

2.8.2 Технические ограничения текущей архитектуры

Таблица 4 ограничения текущей архитектуры

Ограничение	Причина	Влияние на систему
Низкая параллельность записей	SQLite использует блокировку на уровне всей базы (database-level lock) при COMMIT	При >50 одновременных операциях записи возможны задержки или sqlite3.OperationalError: database is locked
Отсутствие асинхронных отчётов	Синхронная генерация оборотных ведомостей в потоке HTTP	Длительные периоды (год) для 10k+ товаров могут вызывать timeout (>30 сек)
Отсутствие внешнего API	Сервис ориентирован на SSR-интерфейс, REST-эндпоинты только для аналитики	Невозможность прямой интеграции с 1С, маркетплейсами, ТСД-сканерами без доработок
Жёсткая привязка к одному серверу	Файловая СУБД и сессии в cookie без распределённого кэша	Сложности горизонтального масштабирования (scale-out) без выноса сессий в Redis

ЗАКЛЮЧЕНИЕ

В ходе выполнения данной курсовой работы была достигнута поставленная цель: спроектирован, реализован и протестирован полнофункциональный веб-сервис для автоматизации складского учёта предприятия. Решение разработано на базе современных открытых технологий (Python 3.x, Flask, SQLAlchemy, SQLite, Jinja2, Vanilla JS/CSS3) и ориентировано на потребности малого и среднего бизнеса, где традиционные корпоративные WMS/ERP-системы экономически нецелесообразны или избыточны.

В процессе работы были последовательно решены все задачи, сформулированные во введении:

Проведён анализ предметной области. Изучены эволюция систем управления складом, современные архитектурные подходы к учёту ТМЦ, принципы документооборота и экономическая эффективность автоматизации. Теоретическое обоснование подтвердило необходимость перехода от табличных и ручных методов к централизованным веб-решениям с журналированием движений.

Выполнено обоснование технологического стека. Сравнительный анализ языков, фреймворков, СУБД и сред разработки позволил выбрать оптимальную комбинацию, обеспечивающую баланс между скоростью разработки, читаемостью кода, безопасностью и нулевой стоимостью лицензирования.

Спроектирована архитектура системы. Реализована клиент-серверная модель с тонким клиентом, модульное разделение ответственности (MVT), паттерн журнального учёта (StockLedger) для гарантированной согласованности остатков, ролевая модель доступа (RBAC) и транзакционное управление операциями. Архитектура заложена с учётом принципов отказоустойчивости и масштабируемости.

Осуществлена программная реализация. Разработан полный цикл складских операций: управление номенклатурой с валидацией уникальности артикулов, создание черновиков накладных, автоматическая нумерация с префиксами, проверка остатков при проведении расхода, отмена проведённых документов, формирование регламентных отчётов («Остатки на дату», «Оборотно-сальдовая ведомость», «Карточка движения»), экспорт в CSV с корректной кодировкой кириллицы, динамическая аналитика (ABC-классификация, график движения) и административный модуль с резервным копированием.

Проведено многоуровневое тестирование. Проверены функциональные сценарии, граничные условия, устойчивость к некорректному вводу, защита от SQL-инъекций и XSS, корректность ролевого разграничения и производительность при последовательных транзакциях. Система продемонстрировала стабильную работу и предсказуемое поведение во всех проверенных режимах.

Практическая значимость разработанного сервиса заключается в его готовности к развёртыванию на реальных объектах без привлечения коммерческого ПО. Использование файловой СУБД SQLite исключает необходимость в выделенном сервере баз данных, а веб-ориентированная архитектура обеспечивает доступ с любых устройств в локальной сети. Совокупная стоимость владения (ТСО) стремится к нулю, а требования к оборудованию минимальны (2 ядра CPU, 2 ГБ ОЗУ, 10 ГБ SSD). Интерфейс построен на принципах прогрессивного улучшения и адаптивной вёрстки, что гарантирует комфортную работу на современном десктопном оборудовании.

Ограничения текущей версии носят архитектурный, а не функциональный характер. SQLite не поддерживает высокий уровень параллельных записей (concurrent writes), что накладывает естественный предел на масштабирование до >100 активных пользователей. Отчёты

генерируются синхронно, что при выборках за многолетние периоды может увеличивать время отклика. В системе отсутствует внешний REST API для интеграции с маркетплейсами, 1С или терминалами сбора данных, а также не реализована поддержка сканеров штрихкодов и термопринтеров «из коробки». Эти ограничения являются осознанным компромиссом в пользу простоты развёртывания и соответствуют уровню MVP для локальных складов МСП.

Перспективы развития проекта чётко структурированы и включают следующие направления:

Внедрение распределённого кэширования через Redis для снижения нагрузки на БД при частых запросах справочников и отчётов.

Добавление полноценного REST API слоя (на базе Flask-RESTful или FastAPI).

Контейнеризация приложения через Docker и Docker Compose для стандартизации сред разработки, тестирования и промышленной эксплуатации.

Разработка PWA-модуля для работы кладовщиков в офлайн-режиме с фоновой синхронизацией операций при восстановлении сетевого подключения.

Цели и задачи достигнуты. Сервис на основе открытых технологий обеспечивает надёжность, безопасность и экономичность складского учёта без коммерческих платформ.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

1. Python Software Foundation. Python 3.12 Documentation [Электронный ресурс]. – Режим доступа: <https://docs.python.org/3/> (дата обращения: 15.04.2026).
2. Pallets Projects. Flask Documentation [Электронный ресурс]. – Режим доступа: <https://flask.palletsprojects.com/> (дата обращения: 15.04.2026).
3. SQLAlchemy Authors. SQLAlchemy 2.0 Documentation [Электронный ресурс]. – Режим доступа: <https://docs.sqlalchemy.org/> (дата обращения: 15.04.2026).
4. SQLite Development Team. SQLite Documentation: Version 3.45+ [Электронный ресурс]. – Режим доступа: <https://www.sqlite.org/docs.html> (дата обращения: 15.04.2026).
5. Pallets Projects. Jinja2 Documentation [Электронный ресурс]. – Режим доступа: <https://jinja.palletsprojects.com/> (дата обращения: 15.04.2026).
6. Гринберг, М. Разработка веб-приложений на Flask / М. Гринберг ; пер. с англ. – 2-е изд. – СПб. : БХВ-Петербург, 2019. – 368 с.
7. Васильев, А. Н. Python. Подробный справочник / А. Н. Васильев. – М. : Эксмо, 2022. – 672 с.
8. Баранов, П. В. Проектирование и разработка веб-приложений на Python / П. В. Баранов. – СПб. : Питер, 2021. – 352 с.
9. OWASP Foundation. OWASP Top 10: 2021. The Ten Most Critical Web Application Security Risks [Электронный ресурс]. – Режим доступа: <https://owasp.org/www-project-top-ten/> (дата обращения: 10.04.2026).
10. NIST. Digital Identity Guidelines: Authentication and Lifecycle Management (SP 800-63B) / National Institute of Standards and Technology. – Gaithersburg : NIST, 2020. – 124 p.
11. Фаулер, М. Архитектура корпоративных программных приложений / М. Фаулер ; пер. с англ. – М. : Вильямс, 2006. – 544 с.

12. Крэйг, У. Складская логистика: теория и практика управления запасами / У. Крэйг. – М. : Инфра-М, 2020. – 288 с.
13. ГОСТ Р 7.0.100–2018. Библиографическая запись. Библиографическое описание. Общие требования и правила составления. – М. : Стандартинформ, 2019. – 138 с.
14. ГОСТ 34.602–89. Техническое задание на создание автоматизированной системы. – М. : Изд-во стандартов, 1989. – 14 с.
15. ISO/IEC 27001:2022. Information security, cybersecurity and privacy protection — Information security management systems — Requirements. – Geneva : ISO/IEC, 2022. – 46 p.
16. Real Python. Flask Tutorial Series: Best Practices and Patterns [Электронный ресурс]. – Режим доступа: <https://realpython.com/tutorials/flask/> (дата обращения: 12.04.2026).
17. Microsoft Learn. Modern Web Application Architecture and Deployment Patterns [Электронный ресурс]. – Режим доступа: <https://learn.microsoft.com/en-us/azure/architecture/web-apps/> (дата обращения: 14.04.2026).
18. Чен, Д. Системы управления складом: от теории к внедрению / Д. Чен. – М. : Альпина Паблишер, 2021. – 312 с.
19. Лутц, М. Изучаем Python. Т. 1 / М. Лутц ; пер. с англ. – 5-е изд. – СПб. : Символ-Плюс, 2020. – 1088 с.